



INFORME DE ESTRUCTURA DE DATOS

I. PORTADA

Tema:	Recorridos de Árboles Binarios
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3B
Alumno participantes	- Amaguaña Shirley
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza

II. INFORME

1.1 Objetivo General

Implementar y analizar los principales recorridos de árboles binarios utilizando C++ y Java, aplicando estructuras de datos dinámicas, recursividad para DFS y colas para BFS, conectando la práctica con los módulos del proyecto final SmartCampus UTA.

1.2 Resultados de Aprendizaje

Al finalizar esta práctica se logró:

- Explicar la diferencia entre recorridos DFS y BFS con un ejemplo real.
- Implementar los recorridos Inorden, Preorden y Postorden usando recursividad.
- Implementar BFS correctamente usando una cola (queue).
- Comparar cómo se implementa lo mismo en C++ y en Java.
- Aplicar los recorridos a los módulos del proyecto final SmartCampus.

Repositorio GitHub: https://github.com/shirley6126/tarea2.2-recorridos_arboles_uta

1.3 Explicación teórica

¿Qué es un árbol binario?

Un árbol binario es una estructura de datos donde cada elemento, llamado nodo, puede tener como máximo dos hijos: uno a la izquierda y otro a la derecha. Todo el árbol parte de un único nodo principal que se llama raíz. Es similar a un árbol real, pero al revés: la raíz está arriba y las ramas bajan hacia abajo.

Los nodos que no tienen hijos se llaman hojas, y los nodos intermedios son los que conectan la raíz con las hojas. Esta estructura es muy útil cuando los datos tienen una relación jerárquica, como por ejemplo los módulos de un sistema de software.

¿Qué es DFS (Búsqueda en Profundidad)?

DFS significa Depth First Search, que en español es búsqueda en profundidad. La idea es que el algoritmo baja lo más que puede por una rama antes de pasarse a la siguiente. Los tres recorridos principales de un árbol (Preorden, Inorden y Postorden) son todos variantes de DFS. La diferencia entre ellos es el momento en que se "visita" o procesa la raíz.

¿Qué es BFS (Búsqueda en Anchura)?

BFS significa Breadth First Search, que en español es búsqueda en anchura o recorrido por niveles. En lugar de bajar profundo por una rama, BFS visita todos los nodos de un nivel antes de



pasar al siguiente. Para poder hacerlo necesita una cola (queue), donde guarda los nodos que todavía le falta visitar.

Diferencia entre DFS y BFS

La diferencia principal está en la dirección del recorrido y en la estructura que cada uno necesita para funcionar:

Característica	DFS (Profundidad)	BFS (Anchura)
Dirección	Baja profundo primero	Va nivel por nivel
Estructura usada	Recursividad (pila)	Cola (Queue FIFO)
Recorridos	Preorden, Inorden, Postorden	BFS / por niveles
Buen para...	Buscar un nodo específico	Ver estructura por niveles
Memoria usada	Proporcional a la altura	Proporcional al ancho

1.4 Explicación de los cuatro recorridos

Preorden (Raíz → Izquierda → Derecha)

Primero se visita la raíz, luego se baja por la rama izquierda, y al final por la derecha. Es útil cuando necesitamos procesar el nodo padre antes que sus hijos, por ejemplo, para copiar un árbol.

Inorden (Izquierda → Raíz → Derecha)

Primero se baja por la izquierda, luego se visita la raíz, y finalmente la derecha. En árboles de búsqueda binaria este recorrido devuelve los elementos en orden ascendente.

Postorden (Izquierda → Derecha → Raíz)

Primero se visitan todos los hijos y al final la raíz. Se usa cuando necesitamos procesar los hijos antes que el padre, como al eliminar un árbol de memoria o calcular el tamaño de carpetas.

BFS - Por niveles (Cola FIFO)

Visita los nodos nivel por nivel usando una cola. Primero la raíz (nivel 0), luego sus hijos directos (nivel 1), luego los nietos (nivel 2), y así. Es útil para encontrar el camino más corto en un grafo o para mostrar una jerarquía organizada.



1.5. Implementación en C++

```
#include <iostream>
#include <string>
#include <queue>

using namespace std;

// Cada nodo guarda el nombre de un módulo del sistema SmartCampus
class Nodo {
public:
    string modulo;
    Nodo *izq, *der;

    // Constructor: cuando creamos un nodo solo le damos el nombre del módulo
    Nodo(string m) : modulo(m), izq(nullptr), der(nullptr) {}
};

// Esta clase solo sirve para mostrar cómo queda el árbol visualmente
class DibujanteArbol {
public:
    // Imprime el árbol en consola con forma de árbol de verdad
    void imprimirEstructura() {
        cout << "\n-- ESTRUCTURA DEL SISTEMA SmartCampus UTA --" << endl;
        cout << "          [SmartCampus Web]          " << endl; // raíz principal
        cout << "          /          \          " << endl;
        cout << "    [Usuarios]      [Academico]      " << endl; // primer nivel de ramas
        cout << "    /      \      /      \      " << endl;
        cout << "  [Auth]  [Roles] [Materias] [Sesiones] " << endl; // segundo nivel
        cout << "    /          \          " << endl;
        cout << "                [Asistencia] " << endl; // hoja más profunda
        cout << "-----" << endl;
    }
};

// Aquí está toda la lógica del árbol y sus recorridos
class GestionSistema {
public:
    Nodo* raiz;

    // Armamos el árbol con los módulos del SmartCampus
    // Los 5 nodos nuevos son: Auth JWT, Roles, Materias, Sesiones y Asistencia
    GestionSistema() {
        raiz = new Nodo("SmartCampus Web"); // raíz del sistema

        // Rama izquierda: todo lo de usuarios
        raiz->izq = new Nodo("Gestion Usuarios");
        raiz->izq->izq = new Nodo("Auth JWT"); // nodo nuevo 1: autenticación
        raiz->izq->der = new Nodo("Roles Docente/Est"); // nodo nuevo 2: roles del sistema

        // Rama derecha: todo lo académico
        raiz->der = new Nodo("Gestion Academica");
        raiz->der->izq = new Nodo("Materias (Vector)"); // nodo nuevo 3: lista de materias
        raiz->der->der = new Nodo("Sesiones (L.Circular)"); // nodo nuevo 4: sesiones activas
        raiz->der->der->izq = new Nodo("Asistencia (cola)"); // nodo nuevo 5: registro de asistencia
    }

    // Preorden: primero visita la raíz, luego va a la izquierda y después a la derecha
    void recorridoPreorden(Nodo* r) {
        if (r == nullptr) return; // si no hay nodo, no hace nada
        cout << "[" << r->modulo << " ] ";
        recorridoPreorden(r->izq); // baja por la izquierda
        recorridoPreorden(r->der); // luego por la derecha
    }

    // Inorden: primero izquierda, luego raíz, luego derecha
    void recorridoInorden(Nodo* r) {
        if (r == nullptr) return;
        recorridoInorden(r->izq);
        cout << "[" << r->modulo << " ] ";
        recorridoInorden(r->der);
    }

    // Postorden: primero los hijos y al final la raíz
    void recorridoPostorden(Nodo* r) {
        if (r == nullptr) return;
        recorridoPostorden(r->izq);
        recorridoPostorden(r->der);
        cout << "[" << r->modulo << " ] "; // la raíz sale de última
    }
};
```



```
// BFS: recorre nivel por nivel usando una cola (queue)
void recorridoBFS() {
    if (raiz == nullptr) return;

    queue<Nodo*> cola; // usamos una cola para ir guardando los nodos pendientes
    cola.push(raiz); // empezamos desde la raíz

    while (!cola.empty()) {
        Nodo* actual = cola.front(); // tomamos el primero de la cola
        cola.pop();

        cout << "[" << actual->modulo << "]" ";

        // Si tiene hijos, los agregamos a la cola para visitarlos después
        if (actual->izq) cola.push(actual->izq);
        if (actual->der) cola.push(actual->der);
    }
};

int main() {
    GestionSistema smart;
    DibujanteArbol dibujo;

    // Primero mostramos cómo se ve el árbol visualmente
    dibujo.imprimirEstructura();

    // Ahora ejecutamos cada tipo de recorrido
    cout << "\nRecorrido Preorden del arbol:";
    cout << "\n(Orden: Raiz -> Izquierda -> Derecha)\n";
    smart.recorridoPreorden(smart.raiz);

    cout << "\n\nRecorrido Inorden del arbol:";
    cout << "\n(Orden: Izquierda -> Raiz -> Derecha)\n";
    smart.recorridoInorden(smart.raiz);

    cout << "\n\nRecorrido Postorden del arbol:";
    cout << "\n(Orden: Izquierda -> Derecha -> Raiz)\n";
    smart.recorridoPostorden(smart.raiz);
```

```
    cout << "\n\nRecorrido Postorden del arbol:";
    cout << "\n(Orden: Izquierda -> Derecha -> Raiz)\n";
    smart.recorridoPostorden(smart.raiz);

    cout << "\n\nRecorrido BFS - por niveles:";
    cout << "\n(Usa una Cola FIFO para ir nivel por nivel)\n";
    smart.recorridoBFS();

    cout << "\n\n";
    return 0;
}
```



1.6. Implementación en Java

```
import java.util.LinkedList;
import java.util.Queue;

// Cada nodo representa un módulo del sistema SmartCampus
class Nodo {
    String modulo;
    Nodo izq, der;

    // Al crear el nodo solo necesitamos el nombre del módulo
    public Nodo(String modulo) {
        this.modulo = modulo;
        this.izq = null;
        this.der = null;
    }
}

// Esta clase dibuja el árbol en consola para verlo más claro
class DibujanteArbol {
    public void imprimirEstructura() {
        System.out.println(x: "\n-- ESTRUCTURA DEL SISTEMA SmartCampus UTA --");
        System.out.println(x: "                [SmartCampus Web]                "); // raíz
        System.out.println(x: "                /                \\" );
        System.out.println(x: "            [Usuarios]            [Academico]            "); // nivel 1
        System.out.println(x: "                /                \\" );
        System.out.println(x: "        [Auth]    [Roles]    [Materias] [Sesiones] "); // nivel 2
        System.out.println(x: "                /                \\" );
        System.out.println(x: "                                [Asistencia] "); // hoja más profunda
        System.out.println(x: "-----");
    }
}

// Aquí está el árbol y todos sus recorridos
class GestionSistema {
    Nodo raiz;

    // Construimos el árbol con los módulos del proyecto SmartCampus
    public GestionSistema() {
        raiz = new Nodo(modulo: "SmartCampus Web"); // raíz del sistema

        // Parte izquierda: gestión de usuarios
        raiz.izq = new Nodo(modulo: "Gestion Usuarios");
        raiz.izq.izq = new Nodo(modulo: "Auth JWT"); // nodo nuevo 1
        raiz.izq.der = new Nodo(modulo: "Roles Docente/Est"); // nodo nuevo 2

        // Parte derecha: gestión académica
        raiz.der = new Nodo(modulo: "Gestion Academica");
        raiz.der.izq = new Nodo(modulo: "Materias (Vector)"); // nodo nuevo 3
        raiz.der.der = new Nodo(modulo: "Sesiones (L.Circular)"); // nodo nuevo 4
        raiz.der.der.izq = new Nodo(modulo: "Asistencia (Cola)"); // nodo nuevo 5
    }

    // Preorden: raíz primero, luego izquierda, luego derecha
    public void recorridoPreorden(Nodo r) {
        if (r == null) return; // si el nodo no existe, salimos
        System.out.print("[ " + r.modulo + " ] ");
        recorridoPreorden(r.izq); // bajamos por la izquierda
        recorridoPreorden(r.der); // luego por la derecha
    }

    // Inorden: izquierda, luego raíz, luego derecha
    public void recorridoInorden(Nodo r) {
        if (r == null) return;
        recorridoInorden(r.izq);
        System.out.print("[ " + r.modulo + " ] ");
        recorridoInorden(r.der);
    }

    // Postorden: primero los hijos, la raíz sale al final
    public void recorridoPostorden(Nodo r) {
        if (r == null) return;
        recorridoPostorden(r.izq);
        recorridoPostorden(r.der);
        System.out.print("[ " + r.modulo + " ] "); // raíz al último
    }

    // BFS: recorre nivel por nivel con ayuda de una cola
    public void recorridoBFS() {
        if (raiz == null) return;

        // LinkedList funciona bien como cola en Java
        Queue<Nodo> cola = new LinkedList<>();
    }
}
```



```
cola.add(raiz); // empezamos desde la raíz

while (!cola.isEmpty()) {
    Nodo actual = cola.poll(); // sacamos el primero de la cola
    System.out.print "[" + actual.modulo + " ] ";

    // Si tiene hijos los metemos a la cola para visitarlos después
    if (actual.izq != null) cola.add(actual.izq);
    if (actual.der != null) cola.add(actual.der);
}

}

// Clase principal donde corre el programa
public class Main {
    Run | Debug
    public static void main(String[] args) {
        GestionSistema smart = new GestionSistema();
        DibujanteArbol dibujo = new DibujanteArbol();

        // Primero mostramos la estructura visual del árbol
        dibujo.imprimirEstructura();

        // Ejecutamos cada recorrido con su explicación
        System.out.print(s: "\nRecorrido Preorden del arbol:");
        System.out.print(s: "\n(Orden: Raiz -> Izquierda -> Derecha)\n");
        smart.recorridoPreorden(smart.raiz);

        System.out.print(s: "\n\nRecorrido Inorden del arbol:");
        System.out.print(s: "\n(Orden: Izquierda -> Raiz -> Derecha)\n");
        smart.recorridoInorden(smart.raiz);

        System.out.print(s: "\n\nRecorrido Postorden del arbol:");
        System.out.print(s: "\n(Orden: Izquierda -> Derecha -> Raiz)\n");
        smart.recorridoPostorden(smart.raiz);

        System.out.print(s: "\n\nRecorrido BFS - por niveles:");
        System.out.print(s: "\n(Usa una Cola FIFO para ir nivel por nivel)\n");
        smart.recorridoBFS();
    }
}
```



1.6. Ejecución en C++ y Java

```
-- ESTRUCTURA DEL SISTEMA SmartCampus UTA --
[SmartCampus Web]
/      \
[Usuarios]  [Academico]
/  \      /  \
[Auth] [Roles] [Materias] [Sesiones]
                        /
                      [Asistencia]
-----

Recorrido Preorden del arbol:
(Orden: Raiz -> Izquierda -> Derecha)
[SmartCampus Web] [Gestion Usuarios] [Auth JWT] [Roles Docente/Est] [Gestion Academica] [Materias (Vector)] [Sesiones (L.Circular)] [Asistencia (Cola)]

Recorrido Inorden del arbol:
(Orden: Izquierda -> Raiz -> Derecha)
[Auth JWT] [Gestion Usuarios] [Roles Docente/Est] [SmartCampus Web] [Materias (Vector)] [Gestion Academica] [Asistencia (Cola)] [Sesiones (L.Circular)]

Recorrido Postorden del arbol:
(Orden: Izquierda -> Derecha -> Raiz)
[Auth JWT] [Roles Docente/Est] [Gestion Usuarios] [Materias (Vector)] [Asistencia (Cola)] [Sesiones (L.Circular)] [Gestion Academica] [SmartCampus Web]

Recorrido BFS - por niveles:
(Usa una Cola FIFO para ir nivel por nivel)
[SmartCampus Web] [Gestion Usuarios] [Gestion Academica] [Auth JWT] [Roles Docente/Est] [Materias (Vector)] [Sesiones (L.Circular)] [Asistencia (Cola)]
```

1.7. Ejecución en Java

```
PS C:\Users\VivoBook\OneDrive\Escritorio\tarea2.2repo_recorridos_arboles_uta\Repo_recorridos_arboles_uta\src\cpp> cd "c:\Users\VivoBook\OneDrive\Escritorio\tarea2.2repo_recorridos_arboles_uta\src\java\" ; if ($?) { javac Main.java } ; if ($?) { java Main }

-- ESTRUCTURA DEL SISTEMA SmartCampus UTA --
[SmartCampus Web]
/      \
[Usuarios]  [Academico]
/  \      /  \
[Auth] [Roles] [Materias] [Sesiones]
                        /
                      [Asistencia]
-----

Recorrido Preorden del arbol:
(Orden: Raiz -> Izquierda -> Derecha)
[SmartCampus Web] [Gestion Usuarios] [Auth JWT] [Roles Docente/Est] [Gestion Academica] [Materias (Vector)] [Sesiones (L.Circular)] [Asistencia (Cola)]

Recorrido Inorden del arbol:
(Orden: Izquierda -> Raiz -> Derecha)
[Auth JWT] [Gestion Usuarios] [Roles Docente/Est] [SmartCampus Web] [Materias (Vector)] [Gestion Academica] [Asistencia (Cola)] [Sesiones (L.Circular)]

Recorrido Postorden del arbol:
(Orden: Izquierda -> Derecha -> Raiz)
[Auth JWT] [Roles Docente/Est] [Gestion Usuarios] [Materias (Vector)] [Asistencia (Cola)] [Sesiones (L.Circular)] [Gestion Academica] [SmartCampus Web]

Recorrido BFS - por niveles:
(Usa una Cola FIFO para ir nivel por nivel)
[SmartCampus Web] [Gestion Usuarios] [Gestion Academica] [Auth JWT] [Roles Docente/Est] [Materias (Vector)] [Sesiones (L.Circular)] [Asistencia (Cola)]
```

1.8. Caso Real: Sistema SmartCampus UTA

El árbol implementado en el código representa la arquitectura modular del proyecto SmartCampus UTA, que es el proyecto final de nuestro curso de Estructura de Datos. Cada nodo del árbol corresponde a un módulo real del sistema:

Nodo	Tipo	Descripción
SmartCampus Web	Raíz	El sistema completo. Es el punto de entrada de todo, de ahí salen todas las demás funcionalidades.
Gestion Usuarios	Rama izquierda	Módulo que maneja todo lo relacionado con los usuarios del sistema (docentes y estudiantes).



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: FEBRERO – JULIO 2026



Auth JWT	Hoja izq-izq (Nuevo 1)	Submódulo de autenticación. Controla el inicio de sesión usando tokens JWT.
Roles Docente/Est	Hoja izq-der (Nuevo 2)	Define si el usuario que ingresa es docente o estudiante y qué puede hacer dentro del sistema.
Gestion Academica	Rama derecha	Módulo que maneja todo lo relacionado con las clases, materias y asistencia.
Materias (Vector)	Hoja der-izq (Nuevo 3)	Lista de materias disponibles. Usa un Vector como estructura interna para guardarlas.
Sesiones (L.Circular)	Hoja der-der (Nuevo 4)	Administra las sesiones de clase activas. Usa Lista Circular para el control de turnos.
Asistencia (Cola)	Hoja más profunda (Nuevo 5)	Registra la asistencia en cada sesión. Usa Cola porque los registros se procesan en el orden en que llegan.

- **¿Por qué se usa un árbol para este caso?**

El sistema SmartCampus tiene una jerarquía natural: el sistema tiene áreas, las áreas tienen módulos, y los módulos tienen submódulos. Un árbol binario refleja exactamente esa relación de padre-hijo. Además, cada recorrido tiene una aplicación práctica concreta:

- Preorden: cargar primero los módulos principales antes que los secundarios (inicio de la aplicación).
- Postorden: liberar módulos desde los más internos hacia afuera sin dejar referencias sueltas.
- Inorden: navegar módulos en un orden específico para mostrar menús organizados.
- BFS: mostrar todos los módulos de un mismo nivel para construir el panel de navegación.

1.9. Uso de GitHub e IA Documentada

Todo el trabajo de la práctica fue versionado y subido al repositorio del grupo en GitHub. El repositorio contiene los siguientes archivos:

Archivo / Carpeta	Contenido
src/cpp/main.cpp	Código C++ completo con comentarios
src/java/Main.java	Código Java completo con comentarios
assets/	Capturas de ejecución en consola (C++ y Java)
README.md	Documentación completa

Se utilizó la herramienta de IA Claude (Anthropic) como apoyo durante el desarrollo de la práctica.

- **Revisión de comentarios del código:** La IA sugirió comentarios más claros y naturales.
- **Estructura del informe:** Se consultó a la IA sobre cómo organizar las secciones.



Conclusiones

1. Los recorridos DFS (Preorden, Inorden, Postorden) se implementan con recursividad porque cada llamada automáticamente recuerda hasta dónde había llegado, lo cual es justo lo que necesitamos para bajar por ramas.
2. El recorrido BFS necesita una cola porque tiene que guardar los nodos que va a visitar después. Sin la cola no se podría mantener el orden nivel por nivel.
3. La diferencia principal entre DFS y BFS no es solo el orden de visita, sino la estructura de datos que cada uno usa internamente.
4. Usar los módulos del SmartCampus como nodos del árbol nos ayudó a entender que los árboles no son solo para números: sirven para representar cualquier cosa con jerarquía.
5. Implementar el mismo código en C++ y Java dejó claro que la lógica es exactamente igual; solo cambia la sintaxis y algunas clases de la biblioteca estándar.

Bibliografía

- [1] B. Stroustrup, The C++ Programming Language, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2013.
- [2] J. Bloch, Effective Java, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2018.
- [3] M. A. Weiss, Data Structures and Algorithm Analysis in C++, 4th ed. Upper Saddle River, NJ: Pearson, 2014.